Ideation Phase

Brainstorm & Idea Prioritization Template

Date	15-07-2026
Team ID	SWTID-20261581
Project Name	Credit Card Approval Prediction System
Maximum Marks	4 Marks

Step 1: Problem Statement

Project: Credit Card Approval Prediction System

How Might We: How might we help financial institutions quickly and fairly predict whether a credit card applicant is a good or bad credit risk, using their demographic, financial and credit-history data?

Step 2: Brainstorm, Idea Listing and Grouping

Ideas generated for the Credit Card Approval Prediction System pipeline, grouped into six clusters:

Data & Features

- Use the Kaggle Credit Card Approval dataset (application_record.csv + credit_record.csv)
- Engineer 'Days Employed' and 'Age' from raw day-counts into readable years
- Derive credit history features: months of EMIs paid off, months overdue, total loan accounts
- Handle missing/outlier income and employment values
- Generate synthetic data as a fallback so the pipeline still runs without the Kaggle files

Risk Labeling

- Map STATUS codes (C, X, 0-5) to a binary risky/not-risky label
- Treat STATUS 2-5 (60+ days overdue, bad debt) as the 'risky' class
- Treat C (paid off), X (no loan that month), 0-1 (early overdue) as low risk

Modelling

- Train and compare Logistic Regression, Decision Tree, Random Forest and XGBoost
- Select the best model by F1-score to handle class imbalance fairly
- Tune the winning model with RandomizedSearchCV
- Persist the trained model and label encoders with pickle (model.pkl, encoders.pkl)

User Experience

- Simple landing page explaining what the tool does
- Single prediction form grouped into Personal, Financial, Family and Credit History sections
- Clear Approved / Rejected result page with a friendly explanation
- Premium dark glassmorphism visual style to feel trustworthy and modern

Deployment & Ops

- Wrap the trained model in a Flask backend with /, /form and /predict routes
- Deploy the app on Render for a free, always-reachable live demo
- Provide a Procfile and render.yaml for one-click cloud deployment
- Keep notebooks (EDA, preprocessing, model training) alongside the production script for transparency

Trust, Fairness & Explainability

- Publish EDA plots (confusion matrices, correlation heatmap, demographic distributions) for transparency
- Avoid using protected attributes as primary decision drivers
- Show applicants which factors most influenced their result
- Add a human-review/appeal path for borderline or rejected cases

Step 3: Idea Prioritization

Ideas placed on the Importance vs. Feasibility grid. Quick Wins (high importance, high feasibility) were tackled first; Major Projects next; Fill-ins as time allowed; Thankless Tasks deprioritized.

Idea	Importance	Feasibility	Priority Quadrant
Compare 4 ML models by F1-score	High	High	Quick Win
Binary risk labelling from STATUS codes	High	High	Quick Win
Flask form + result pages	High	High	Quick Win
Deploy live demo on Render	High	Medium	Quick Win
Feature engineering (age/employment/credit history)	High	Medium	Major Project
RandomizedSearchCV tuning of best model	Medium	Medium	Major Project
Explainability / factor breakdown for applicants	High	Low	Major Project
Glassmorphism UI polish	Medium	High	Fill-in
Synthetic data fallback generator	Medium	High	Fill-in
IBM Watson ML cloud deployment	Low	Low	Thankless Task